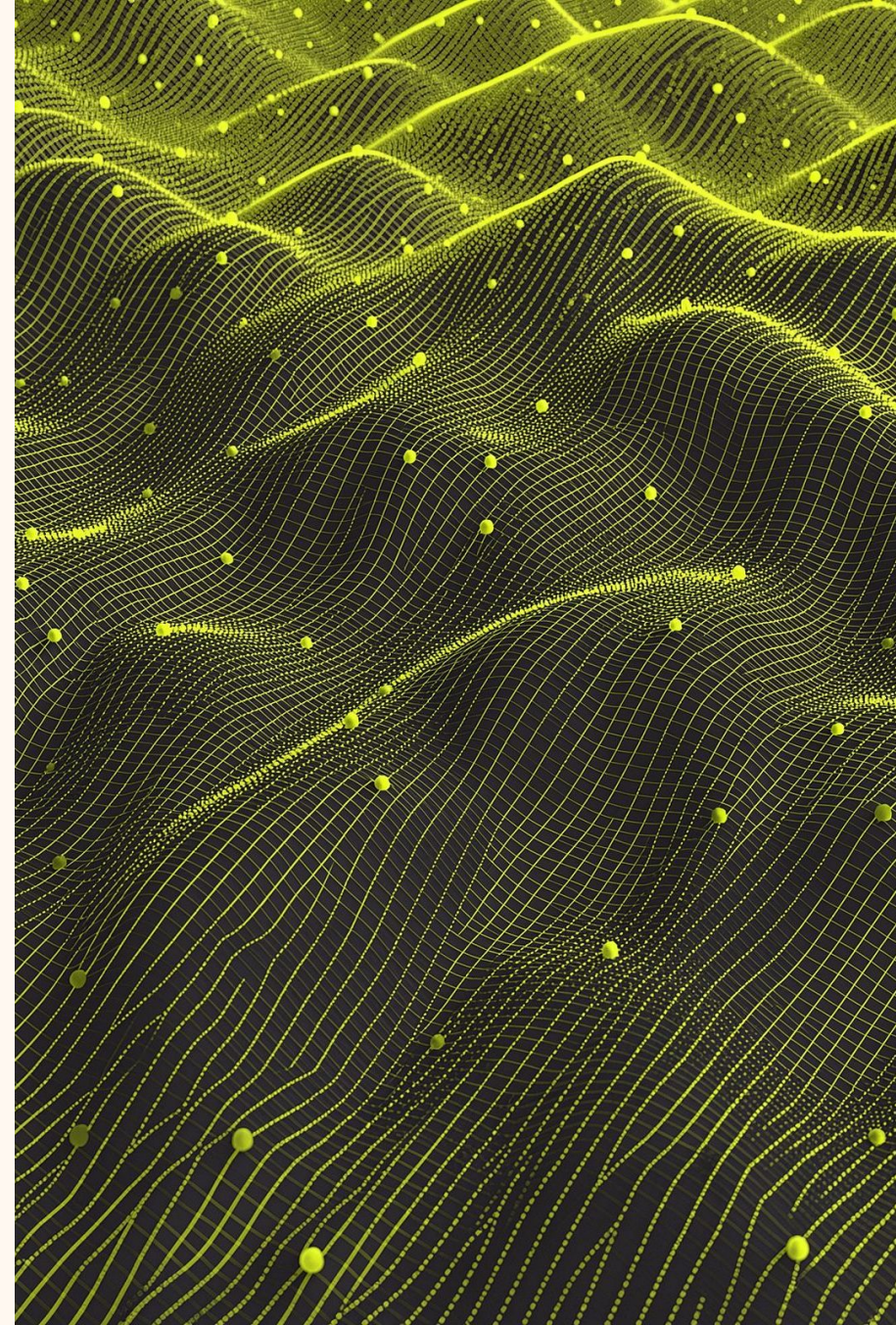


Credit Risk Assessment in Banking

Model Evaluation Metrics | Classification Model for Loan Default Prediction

Dr. Anilkumar V Brahmane | Computer Engineering | Sanjivani College of Engineering



What is Credit Risk?

Credit Risk

The possibility that a borrower will fail to repay a loan — causing financial loss to the bank.

Why It Matters

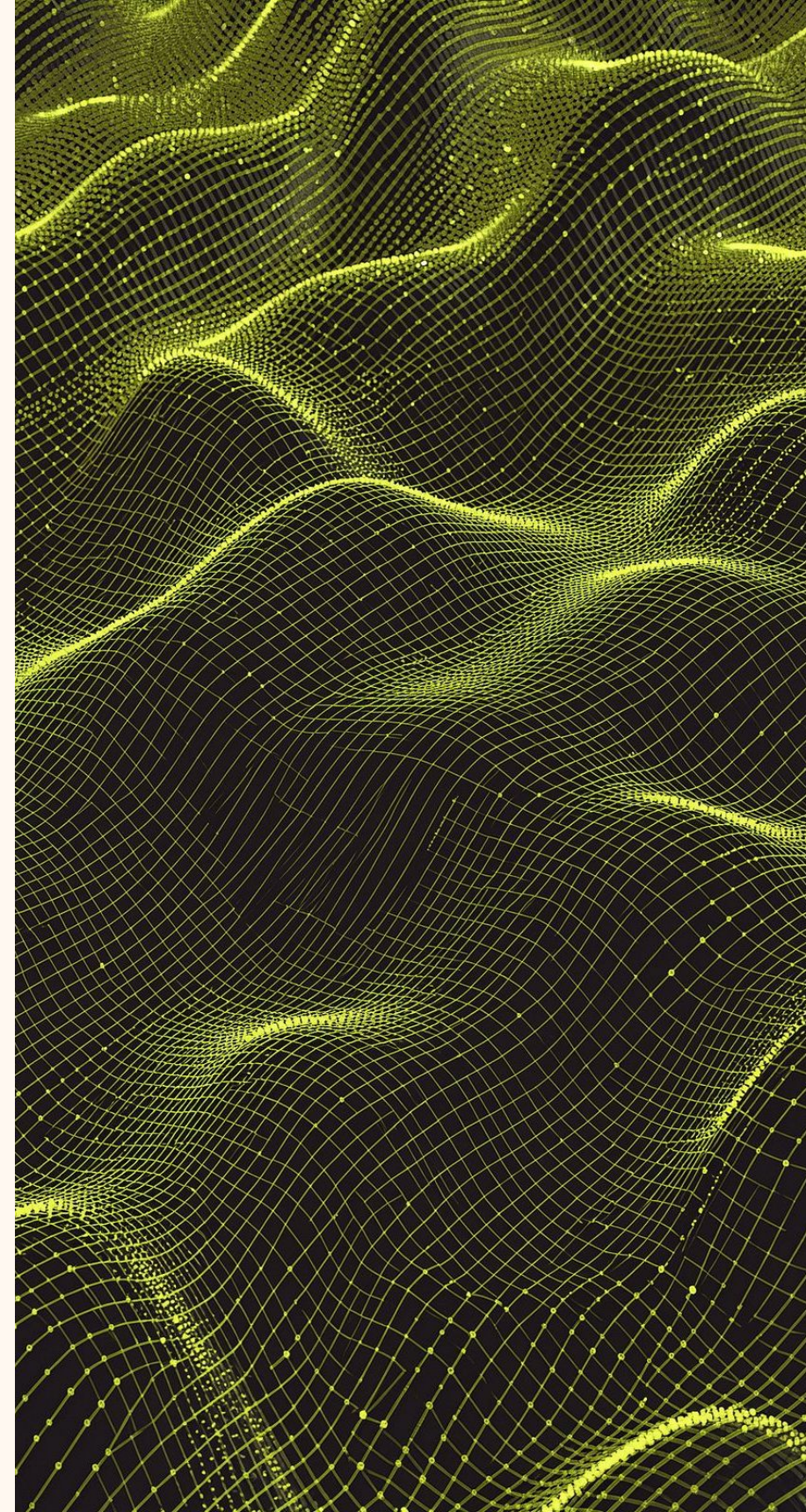
Banks lose billions annually due to loan defaults. Accurate prediction saves money and protects customers.

Our Goal

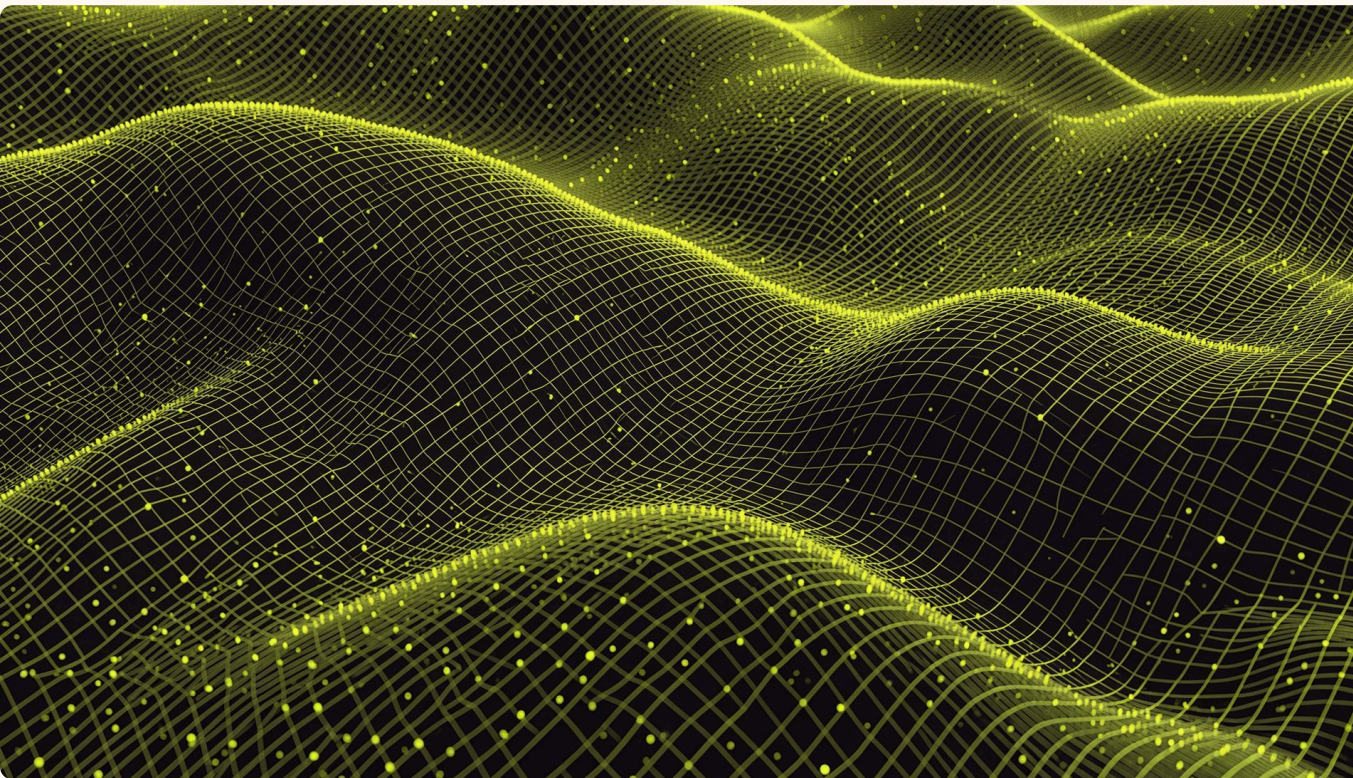
Build a Python ML model that predicts — **Will this person default on their loan? (Yes / No)**

Real-World Impact

Used by banks like HDFC, SBI, and global lenders to approve or reject loan applications automatically.



Step 1 — Dataset & Problem Setup



Dataset Features

- Age, Income, Loan Amount, Credit Score
- Employment Duration, Debt-to-Income Ratio (DTI)

Target Variable

default → **1** = Defaulted (did NOT repay) |
0 = Repaid successfully

Class Imbalance

Only ~10–15% of loans default — the model must learn from imbalanced data.

Python Code

```
import pandas as pd      # Load  
and handle tabular data  
df = pd.read_csv('loan_data.csv') #  
Read the dataset file  
df['default'].value_counts() # Check  
defaults vs. repaid
```

Step 2 — Data Preprocessing



Handle Missing Values

Fill or drop rows with null/NaN values so the model doesn't crash during training.



Feature Scaling

Use **StandardScaler** to normalize numbers — e.g., Income (₹50,000) and Age (30) are on very different scales.



Train-Test Split

Divide data — **80%** for training the model, **20%** for testing its accuracy.

```
from sklearn.preprocessing import StandardScaler
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42) # Split data 80/20
scaler.fit_transform(X_train) # Scale training features
scaler.transform(X_test)     # Scale test features (same scale)
```

Step 3 — Build the Classification Model 🤖

Algorithm: Random Forest Classifier

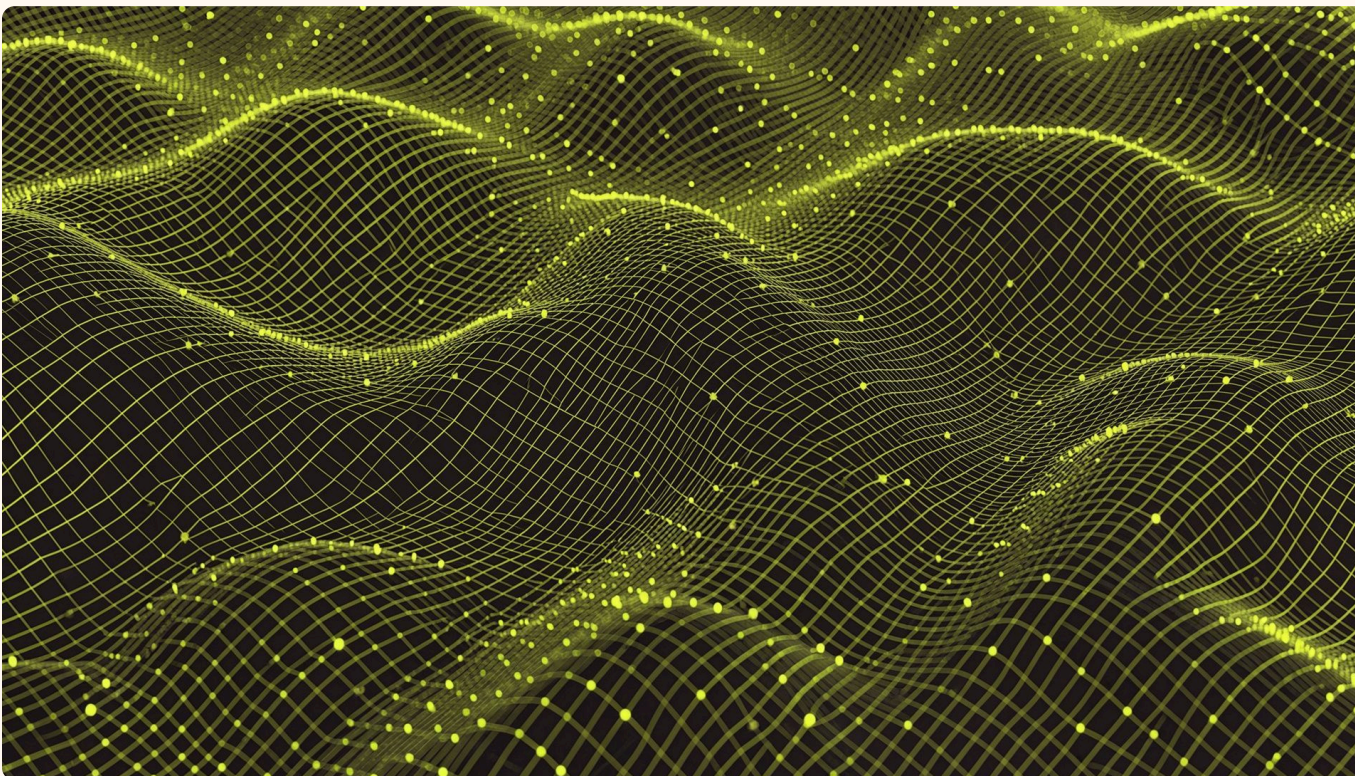
An **ensemble of many decision trees** that vote on the final prediction — the majority wins!

Why Random Forest?

- Handles imbalanced data well
- Robust to outliers and noise
- Gives feature importance scores

Python Code — Step by Step

```
from sklearn.ensemble import  
RandomForestClassifier  
  
# Create model with 100 decision  
trees  
model = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42)  
  
model.fit(X_train, y_train) # Train the  
model  
y_pred = model.predict(X_test) #  
Predict on unseen data
```



Step 4 — Confusion Matrix Explained

The Four Outcomes

✓ True Positive (TP)

Predicted Default →
Actual Default. Correct!

✓ True Negative (TN)

Predicted No Default →
Actual No Default.
Correct!

✗ False Positive (FP)

Predicted Default →
Actually Repaid. Bank
rejects a good customer!

✗ False Negative (FN)

Predicted No Default →
Actually Defaulted. Bank
loses money!

Python Code

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Returns a 2x2 matrix
cm = confusion_matrix(y_test, y_pred)

# Visualize as a color heatmap
sns.heatmap(cm, annot=True, fmt='d',
            cmap='Blues',
            xticklabels=['No Default', 'Default'],
            yticklabels=['No Default', 'Default'])
plt.title('Confusion Matrix')
plt.show()
```

- ❏ The **False Negative** is the most dangerous outcome for banks — a missed default directly causes financial loss.

Step 5 — Accuracy, Precision & Recall



Accuracy

$$= (TP + TN) / \text{Total}$$

Overall % of correct predictions. *Misleading when data is imbalanced!*



Precision

$$= TP / (TP + FP)$$

Of all predicted defaults, how many were real? Avoids false alarms on good customers.

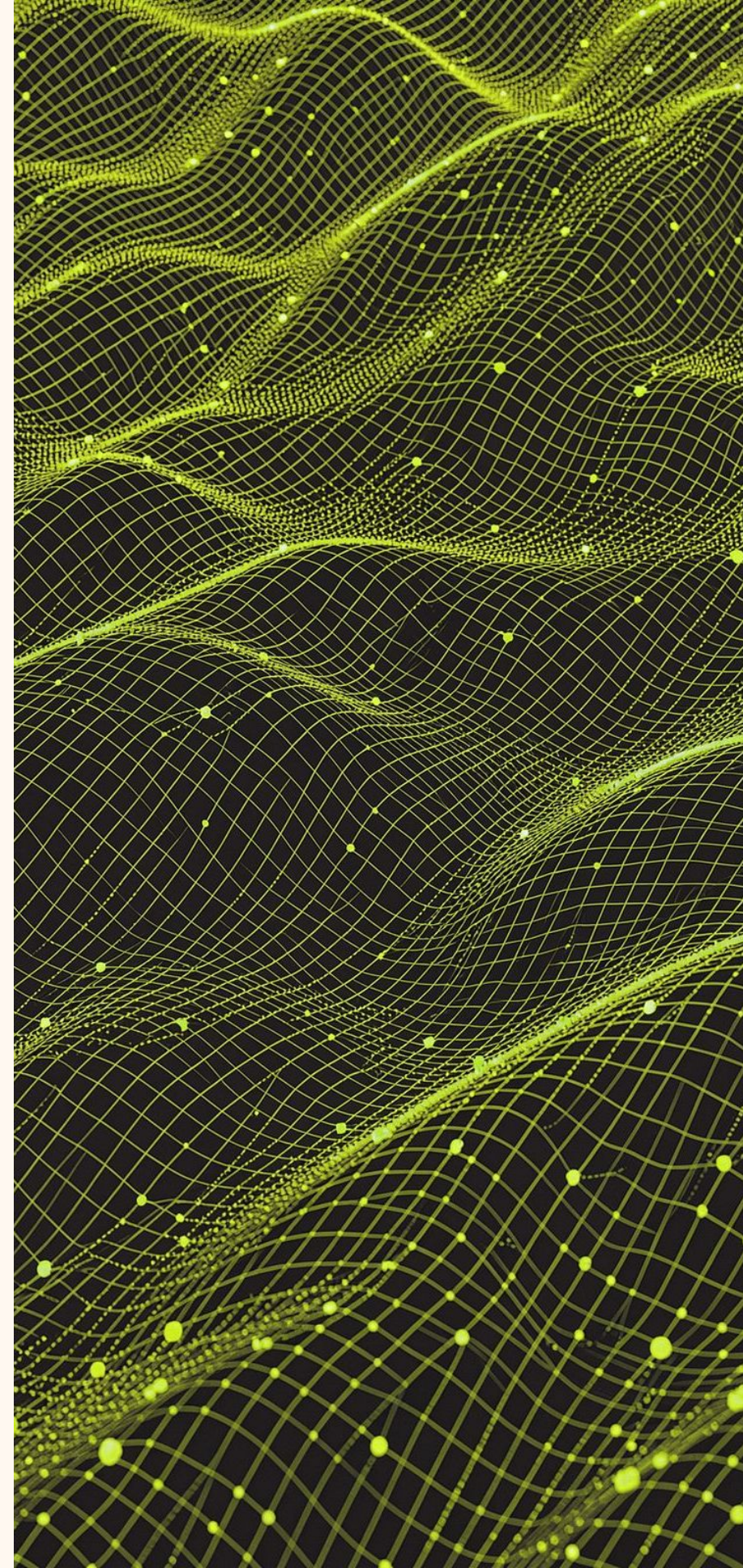


Recall

$$= TP / (TP + FN)$$

Of all actual defaults, how many did we catch? Critical for banks — missing a default is costly!

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score
print("Accuracy: ", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall: ", recall_score(y_test, y_pred))
```



Step 6 — F1-Score: The Balanced Metric

The Problem

High Precision may mean low Recall and vice versa — we need **one single balanced score**.

F1-Score Formula

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

The **harmonic mean** of Precision and Recall. Best value = **1.0** (perfect) | Worst = **0.0**.

Use F1 when **both false positives and false negatives** matter to your business.

Python Code

```
from sklearn.metrics import f1_score,
classification_report

print("F1-Score:", f1_score(y_test,
y_pred))

# Prints Precision, Recall, F1 for ALL
classes at once
print(classification_report(y_test,
y_pred))
```

Step 7 — ROC Curve & AUC Score

What is the ROC Curve?

Plots **True Positive Rate** vs. **False Positive Rate** at every possible decision threshold — giving a full picture of model performance.

AUC — Area Under Curve

Why use it? Works well even with imbalanced datasets and shows model performance across all thresholds.

Python Code — ROC Curve

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

# Get probability scores (not just 0 or 1)
y_prob = model.predict_proba(X_test)[:, 1]

# Compute curve data points
fpr, tpr, _ = roc_curve(y_test, y_prob)

auc = roc_auc_score(y_test, y_prob)

# Plot the ROC curve
plt.plot(fpr, tpr, label=f'AUC = {auc:.2f}',
         color='darkorange', lw=2)
plt.plot([0,1],[0,1], 'k--') # Diagonal = random
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```

AUC = 1.0

Perfect model 🏆

AUC = 0.5

Random guessing 🎲

Summary — Complete Python Pipeline 🚀

1

Step 1: Load

`pd.read_csv()`

2

Step 2: Preprocess

`StandardScaler` + `train_test_split`

3

Step 3: Train

`RandomForestClassifier` `.fit()`

4

Step 4: Evaluate

Confusion Matrix · Accuracy · Precision · Recall · F1 · ROC-AUC



Key Takeaway: Never rely on accuracy alone for imbalanced banking data — always use **Precision, Recall, F1, and ROC-AUC together** for a complete picture of model performance.